Cartpole 闭环实验说明

概述

这个目录现在包含一套基于论文思路改写的 `cartpole` 闭环流程，作为当前主闭环实现使用。它不再依赖原来的 `pong` 任务逻辑。

整个运行链路分成两部分：

- `cartpole_setup.py` 负责初始化 MaxLab、布线电极、连接 stimulation unit、生成 encoding 和 adaptive training 所需的 stimulation sequence、导出运行时 JSON 配置、启动 recording，并拉起 C++ 实时闭环程序。
- `maxone_with_filter.cpp` 负责实时 closed loop。它会读取运行时 JSON 配置，解码两个输出通道，运行 cartpole 环境，发送 encoding stimulation pulse，在 episode 失败后按条件触发 adaptive training sequence，并写出 episode 日志。

当前实现遵循的核心参数如下：

- `200 ms` read window
- `300 ms` training window
- cartpole terminal angle `|theta| > 16 deg`
- rate coding 参数 `a = 7`, `b = 0.15`
- EMA 参数 `alpha = 0.2`
- 输出 force 限制在 `[-10 N, 10 N]`
- 当前只接入 `adaptive training`
- 运行模式支持：
  - `cycled_adaptive`
  - `continuous_adaptive`

相关文件

- cartpole_setup.py
- maxone_with_filter.cpp
- cartpole_task.cpp
- training_controller.cpp
- gamewindow.cpp
- Makefile

运行前提

你需要先具备以下环境：

- 可正常导入 `maxlab` 的 Python 环境
- 当前 shell 中可用的 C++ compiler
- `maxlab_lib/maxlab` 下已经存在可用的 header 和 library
- 可访问 MaxOne / MaxLab 硬件

说明：

- 我写这套代码时，当前工作区里没有可用的 `g++`、`cl` 或 `clang++`，所以代码已经完成，但我无法在这里实际完成 C++ 编译验证。
- Python 入口脚本已做过语法检查，可以正常通过 `py_compile`。

编译

在仓库根目录下执行：

```
cd maxlab_lib
make maxone_with_filter
```

如果你不想启用 Qt GUI，可以关闭 `USE_QT`：

```
cd maxlab_lib
make USE_QT=0 maxone_with_filter
```

编译成功后，目标可执行文件应位于：

- `maxlab_lib/build/maxone_with_filter`

运行前必须检查的电极配置

当前版本仍然是一个“人工指定 unit 的过渡版”。所以在真正运行实验前，你需要先检查并根据自己的芯片/样本情况修改 cartpole_setup.py 顶部这些常量：

- `ALL_STIM_ELECTRODES` 总 stimulation electrode 池
- `ENCODING_STIM_ELECTRODES` 前 2 个 stimulation electrode，作为论文中的两个 input unit
- `TRAINING_STIM_ELECTRODES` 剩余 stimulation electrode，用于构造 adaptive training pair
- `DECODING_LEFT_ELECTRODES` 手动指定的左侧 output unit 对应 recording electrode
- `DECODING_RIGHT_ELECTRODES` 手动指定的右侧 output unit 对应 recording electrode

注意：

- 当前实现默认必须有且只有 2 个 encoding stimulation unit。
- adaptive training 至少需要 2 个可用的 training stimulation unit。
- decoding 目前仍是人工指定，不是论文中完整的 `Record -> Stimulate -> unit identification -> connectivity selection` 自动流程。

Python 脚本实际做了什么

运行 `cartpole_setup.py` 时，会按顺序执行以下步骤：

1. 初始化 MaxLab，并打开 stimulation power。
2. 激活指定的 wells。
3. 对 recording electrode 和 candidate stimulation electrode 做 routing。
4. 为 encoding 和 training electrode 解析 stimulation unit 映射。
5. 将 array configuration 下载到芯片。

6. 执行 offset compensation，并清空 event buffer。
7. 打开并配置所有已解析出的 stimulation unit。
8. 生成 persistent stimulation sequence：
    - `encode_left_pulse`
    - `encode_right_pulse`
    - `train_pair_i_j`
9. 导出供 C++ 使用的运行时 JSON 配置。
10. 启动 data recording。
11. 启动 C++ 进程，并通过 stdin 发送 `start`。
12. 等待 C++ 进程退出。
13. 停止 recording 并清理资源。

运行时配置文件

Python 脚本每次运行都会生成一个 runtime JSON：

- `~/cartpole_experiments/cartpole_<mode>_<timestamp>_config.json`

这个 JSON 会被 `maxone_with_filter` 直接读取。它包含：

- target well
- read / training window 参数
- 实验总时长
- cycle / rest 参数
- spike detector 参数
- decoding channel 列表
- encoding sequence 名称
- adaptive training sequence 名称
- episode 日志输出路径

通常不需要手动改这个 JSON，它是每次运行自动生成的。

如何运行

在仓库根目录下运行：

```
python maxlab_lib/closedloop/cartpole_setup.py --duration 15 --mode
cycled_adaptive
```

如果你要跑 continuous adaptive：

```
python maxlab_lib/closedloop/cartpole_setup.py --duration 15 --mode
continuous_adaptive
```

如果你想打开 Qt viewer：

```
python maxlab_lib/closedloop/cartpole_setup.py --duration 15 --mode
cycled_adaptive --show-gui
```

CLI 参数

- `--duration` 实验总时长，单位分钟。这个值会写入 runtime config，并由 C++ loop 强制执行。
- `--mode` 可选值：
  - `cycled_adaptive`
  - `continuous_adaptive`
- `--wells` 要激活的 well 列表，默认是 `[0]`
- `--show-gui` 如果二进制是按 `USE_QT` 编译的，则会打开 Qt cartpole viewer

C++ 实时闭环里的行为

在 `maxone_with_filter.cpp` 中，每个 timestep 的逻辑如下：

- 每 `200 ms` 读取一次 spike count
- 从 `decoding_left_channels` 和 `decoding_right_channels` 汇总 output activity
- 将这两个 output activity 做 EMA smoothing
- 根据 firing-rate difference 计算 cart force
- 推进一步 cartpole environment
- 根据当前 `theta` 计算两个 input 的 stimulation frequency
- 按需要回放：
  - `encode_left_pulse`
  - `encode_right_pulse`

Episode 何时结束

当满足以下条件时，episode 结束：

- `|theta| > 16 deg`

结束后会计算：

- 当前 episode 的 duration
- 最近 5 个 episode 的 moving mean
- 最近 20 个 episode 的 moving mean

Adaptive training 何时触发

只有在以下条件同时满足时才会触发 training：

- 至少已经积累了 5 个 episode
- `mean_5 <= mean_20`

一旦触发：

- adaptive controller 会采样一个 `train_pair_i_j`
- training window 持续 `300 ms`

- 在这个时间窗内，只发送 training，不发送 encoding

两种模式的区别

`cycled_adaptive`

系统会在 active cycle 和 rest 之间切换：

- active cycle: `15 min`
- rest: `45 min`

`continuous_adaptive`

系统持续保持 active，不进入 rest。

输出文件

默认输出目录：

- `~/cartpole_experiments`

每次运行会生成以下文件：

- `cartpole_<mode>_<timestamp>.raw.h5` MaxLab recording 文件
- `cartpole_<mode>_<timestamp>_config.json` C++ 运行时使用的配置文件
- `cartpole_<mode>_<timestamp>_episodes.jsonl` C++ 写出的逐 episode 日志

Episode 日志字段

当前每一行 JSON 至少包含：

- `episode_index`
- `time_balanced_s`
- `mean_5_s`
- `mean_20_s`
- `training_delivered`
- `training_sequence`
- `terminal_theta_rad`

Stimulation sequence 说明

Encoding sequence

包括：

- `encode_left_pulse`
- `encode_right_pulse`

每个 encoding sequence 都会：

- 连接目标 stimulation unit
- 断开其他 stimulation unit

- 发送一个 biphasic pulse

Adaptive training sequence

每个 `train_pair_i_j` 都表示一个 training unit pair。

每个 sequence 会：

- 选择一个 training pair
- 发送 paired biphasic pulse
- 使用 `5 ms` inter-pulse interval
- 以 `10 Hz` 重复
- 总时长 `300 ms`

当前实现的限制

这套实现是从旧的 `pong` 代码迁移到论文风格 `cartpole` 的中间版本，还不是论文的完整端到端复现。

当前限制包括：

- encoding / decoding / training unit 仍然是手动指定
- 还没有实现论文里的完整 `Record -> Stimulate -> unit identification -> connectivity selection` 自动化流程
- `null` 和 `random` 两种 training paradigm 还没有接入当前 runtime
- C++ 中的 JSON parser 是按当前生成格式写的轻量实现，不适合通用 JSON 输入
- 由于当前 shell 没有 compiler，我无法在这里完成真正的 build verification

建议的首次验证流程

建议你按这个顺序验证：

1. 先编译 `maxone_with_filter`
2. 先跑一个很短的实验，例如 `--duration 1`
3. 确认 C++ 进程能输出 sync 日志和 episode 日志
4. 确认 `*_config.json` 和 `*_episodes.jsonl` 正常生成
5. 确认 decoding channel 上能读到 activity，Qt viewer 中 cartpole 状态会变化
6. 再开始跑 `15 min` 的 `cycled_adaptive` 或 `continuous_adaptive`

下一步建议

如果你要继续把它推进到更接近论文复现，建议按这个顺序往下做：

- 把人工指定的电极常量抽成外部配置文件
- 实现论文里的 automatic unit-selection pipeline
- 接入 `null` 和 `random` paradigm
- 增加 cycle-level 90th percentile performance 分析
- 针对你本机的 compiler / Qt / MaxLab 环境补一份本地 build 文档